TEST PLAN

Runtime Terror

Runtime Terror:

Carlos Astudillo
Jacob Lawler
James Odjewuyi

Mekhi Prospere
Kevin Kangethe

Student Academic Progress Tracker

Runtime Terror
Midwestern State University
Wichita Falls, TX - 76308

1. INTRODUCTION

1.1. Statement of Purpose

The purpose of this document is to define the testing approach for the Student Academic Progress Tracker developed by Runtime Terror. This document establishes the testing scope, methods, environments, and procedures to verify the system's correctness, reliability, and usability before final delivery.

1.2. Overview of the Product

The Student Academic Progress Tracker is a single user desktop advising application designed for Midwestern State University Computer Science students. The system allows students to monitor degree progress, calculate GPA, evaluate prerequisite completion, and determine eligible future coursework through a browser-based interface connected to a local C++ backend server.

1.3. Test types

1.3.1. Unit testing

Unit testing verifies the correctness of individual C++ classes, methods, and frontend functions independently of the rest of the system.

Primary unit testing targets will include:

- GPA calculations
- prerequisite checking
- CSV parsing
- JSON serialization
- file loading and saving
- alert generation
- degree requirement validation

White-box testing techniques will be used to verify internal logic paths and edge cases

1.3.2. Integration testing

Integration testing verifies communication and interaction between:

- frontend JavaScript and backend routes
- HTTP server and C++ objects
- DegreeAudit and DegreePlan
- persistence system and Student objects

Testing ensures that data flows correctly between all components and that route handling produces correct responses.

white-box testing methods will be primarily used during this phase

1.3.3. System testing

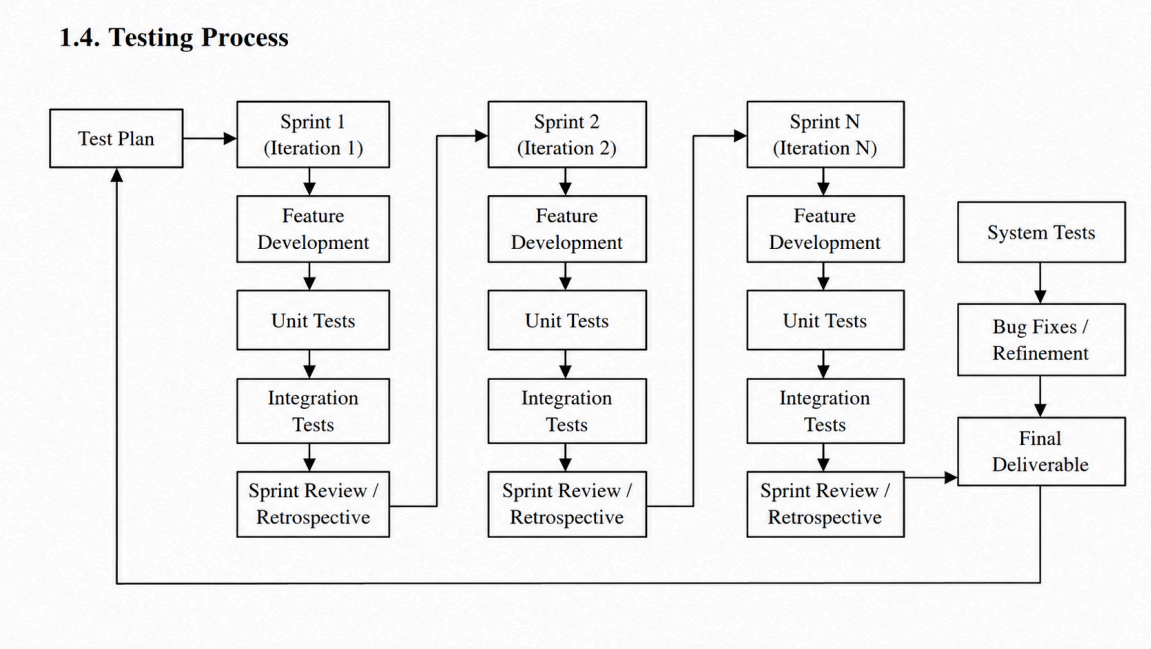
System testing validates the complete application running in its intended environment on a Windows machine.

Testing will verify:

- application startup
- browser launch
- dashboard functionality
- degree audit behavior
- autosave functionality
- user workflow correctness
- recovery after restart

The complete system will be tested using realistic user interactions and academic scenarios.

1.4. Testing Process



2. SCOPE AND OBJECTIVES

As mentioned in the requirements document, the Student Academic Progress application is designed to allow students to track their academic progress and monitor degree completion. This application will focus strictly on academic progress tracking. Outlined below are the main test types that will be performed. All test plans and conditions will be developed from the requirements document and project design document.

2.1. Unit testing

At agreed upon regularly scheduled intervals, the system's developmental progress will be subject to peer review. The testing schedule will correspond to the completion of individual classes, routes, or frontend modules. The objective of these reviews is to ensure correctness and to test the functional integrity within each individual module. Issues to consider include parameter matching, JSON serialization, file input/output, GPA calculations, prerequisite validation, route handling, and memory management.

- Course

- AcademicRecord
- Alert
- Student
- DegreePlan
- DegreeAudit
- Server routes
- Frontend JavaScript functions

Specific methods to be tested include:

- Course::fromCSV()
- Course::loadFromFile()
- AcademicRecord::getGradePoints()
- Student::calculateGPA()
- Student::addRecord()
- DegreeAudit::checkRigidCourses()
- DegreeAudit::checkPrereqs()
- DegreeAudit::findEligibleCourses()
- saveStudentToFile()
- loadStudentFromFile()

Entrance Criteria – At least one module or class should be coded and compiled successfully before formal review and unit testing can begin. As the group meets regularly throughout the week, the development team will attempt to complete at least one major module or route between meetings so that it can be formally reviewed during the next testing session.

Exit Criteria – All errors identified during formal reviews and unit testing are fixed and tested.

2.2. Integration testing

This test proves that all areas of the system interface with each other correctly and that there are no gaps in data flow between the frontend, backend, and persistence system. Final integration testing proves that the application functions correctly as an integrated unit after all fixes are complete.

The actual testing method used for this phase will primarily utilize the Black box testing method. Bottom-up testing strategy will be followed throughout the integration testing phase.

Integration testing will focus on communication between:

- HTML/CSS/JavaScript frontend
- JavaScript fetch() requests
- cpp-httplib backend processing
- DegreeAudit and DegreePlan interaction
- Student and AcademicRecord interaction
- Local file persistence system

The following functionality will be tested during this phase:

- Browser communication with localhost:8080
- HTTP GET, POST, PUT, and DELETE routes
- JSON request and response handling
- Dashboard rendering using returned audit data
- Automatic GPA recalculation after record changes
- Automatic save functionality after data modification
- Loading student data after restart
- Degree audit generation and frontend display

Entrance Criteria – Enough code must be developed and unit tested to complete at least one integrated subsystem involving frontend communication and backend processing.

Exit Criteria –All high-priority errors identified during Integration testing must be fixed and tested. All remaining low priority errors that are not fixed should be documented for future review.

2.3. System testing

This test intends to prove that the functionality delivered by Runtime Terror is as specified in the requirements and design documents. It also assesses the quality, reliability, and usability of the Student Academic Progress Tracker system and ensures that the software successfully supports the intended advising and degree audit functions.

The testing strategy for validation of the system as a whole will also utilize the black box method. All possible user input must be examined, and any deficiencies addressed. Like the integration testing phase, testing will be performed in a bottom-up manner.

System testing will validate the complete runtime environment including:

- Application executable startup
- Automatic browser launch
- Frontend dashboard functionality
- Course addition, update, and deletion
- GPA calculation accuracy
- Degree audit generation
- Prerequisite validation
- Eligible course recommendations
- Alert generation and display
- File persistence and autosave

Testing will be performed using realistic Computer Science student academic records based on Midwestern State University degree requirements.

Entrance Criteria – All modules and classes must be implemented, unit tested, and integration tested. Frontend and backend communication must function correctly and the system must operate in its intended Windows environment.

Exit Criteria – All high-priority errors from the system test must be fixed and tested. If any low-priority errors remain unfixed, they should be documented.

3. TEST SCHEDULE

For the testing phases, the following schedule will apply, with room for flexibility as needed:
Unit testing: 7 days
Integration testing: 5 days
System testing: 3 days

4. RESOURCES

4.1. Human

The Runtime Terror development team consists of:

Carlos Astudillo

Jacob Lawler

James Odjewuyi

Mekhi Prospere

Kevin Kangethe

All members of the team will participate in formal reviews, unit testing, integration testing, and system testing activities throughout the development cycle.

4.2. Hardware

The testing phases will require at least onePC with the following specifications:

- 64bit architecture
- Dual-core processor at 1.5 GHz or higher
- Minimum 4 GB RAM
- At least 200 MB of available disk space
- Keyboard, mouse, and monitor
- An Internet browser capable of running JavaScript

4.3. Software

System testing requires the following software:

- Windows 10 or later operating system
- Visual Studio 2026
- C++ compiler with C++17 support
- Modern web browser
- cpp-httplib library

5. RECORDING PROCEDURES

During Unit testing, Integration testing, and System testing, errors will be recorded as they are detected on Error Report forms. Errors agreed upon as valid by the team will be categorized as high-priority errors and low-priority errors based on their impact on system functionality, data integrity, and usability. All identified errors during reviews and testing phases will be documented in a formally approved format for reporting purposes.

5.1. Error Report Forms

A sample error report form that will be used to record detected errors during Unit testing is given in appendix 6.1. A sample error report form used during Integration testing is given in appendix 6.2 and a sample error report form used during System testing is given in appendix 6.3.

5.2. Unit test results

All the errors recorded during unit tests of each prototype along with the action taken to correct the error will be documented as a Unit test results document.

5.3. Integration test results

All the errors recorded during the integration test phase of each prototype along with the category of the error, status of the error and action taken will be documented as an Integration test results document.

5.4. System test results

All the bugs identified while applying each of the following test cases during System test along with category of the error, status of the error and action taken to correct the error will be documented as a System test results document.

5.4.1. System test cases

The following test cases were identified from the requirements and design documents:

- First-time application startup
- Automatic browser launch
- Creating a new student profile
- Loading existing student data
- Adding a completed course
- Updating a course grade
- Removing a course record
- GPA recalculation after record changes
- Degree audit generation
- Fulfilled course detection
- Unfulfilled course detection
- Prerequisite validation
- Eligible course recommendation generation
- General education category assignment
- Preventing duplicate course reuse between categories
- Alert generation and display
- Autosave after data modification
- Loading saved data after restart
- Invalid course code handling
- Missing file recovery handling
- Route response validation
- Frontend dashboard rendering
- Graceful application shutdown
- continuation RAF due to inoperable condition of a rented car.
- Service agent checking the service list.
- Service agent updating service record of a vehicle.
- Service agent updating Gas report.
- Manager adding/deleting an employee.
- Manager adding/deleting a vehicle.
- Manager updating rental rates.

5.5. Non-fixed errors report

All errors that are left not fixed will be reported with its category, chances of occurrence, action taken and possible remedy in case of occurrence in a separate Non-fixed errors report.

6. APPENDICES

6.1. Unit test error report form

UNIT TEST ERROR REPORT FORM
CARRENTAL SOFTWARE
MAVIS TEAM

	e #	Module	Class

Problem summary:

Reported by

Action taken:

Action taken by

6.2. Integration test error report form

INTEGRATION TEST ERROR REPORT FORM
CARRENTAL SOFTWARE
MAVIS TEAM

Class Category Status

Problem summary:

Reported by

Action taken:

Action taken by

6.3. System test error report

SYSTEM TEST ERROR REPORT FORM
CARRENTAL SOFTWARE
MAVIS TEAM

Class Category Status

Test case:

Problem summary:

Reported by

Action taken:

Action taken by

6.4. Glossary of Terms

Black-box testing – the tester is concerned only with functionality, including system inputs and outputs, not with implementation details.

Bottom-up testing – Integrating and testing lower-level components first before testing higher-level modules.

Encapsulation – Packaging data and operations into a single programming unit.

High-priority error – Serious errors that prevent the correct operation of the system or major functionality.

Low-priority error – Minor errors that do not significantly affect overall functionality.

White box testing – Testing method in which the tester examines internal code structure and logic paths.

Localhost server – A server running locally on the user's machine using localhost:8080.

JSON – JavaScript Object Notation is used for communication between the frontend and backend.

Autosave – Automatic writing of student data to local storage after modifications are made.